SDE BACKEND INTERVIEW SERIES

Kafka & Asynchronous Communication

Interview Questions & Answers | Basics to Advanced

Async Patterns · Kafka Internals · Producers & Consumers · Rebalancing · Exactly-Once & Transactions · Streams · Connect · Schema Registry · Performance & Ops · Saga · Outbox · CQRS · System Comparisons · Scenarios

Jaydip Dey SDE Backend · Java · Kafka · Distributed Systems

Contents

Section 1: Asynchronous Communication Fundamentals	Q1–Q12
Section 2: Kafka Fundamentals	Q13–Q24
Section 3: Kafka Architecture & Internals	Q25–Q36
Section 4: Producer Deep Dive	Q37–Q45
Section 5: Consumer Deep Dive & Rebalancing	Q46–Q55
Section 6: Exactly-Once, Transactions & Delivery Semantics	Q56–Q61
Section 7: Kafka Ecosystem — Streams, Connect, Schema Registry	Q62–Q68
Section 8: Operations, Performance & Tuning	Q69–Q76
Section 9: Architecture Patterns with Kafka	Q77–Q82
Section 10: Kafka vs Other Systems	Q83–Q87
Section 11: Scenario-Based & Behavioral-Style Questions	Q88–Q95

Total: **95 questions** spanning asynchronous communication fundamentals, Kafka architecture and internals, producer/consumer tuning, exactly-once semantics, the Kafka ecosystem, operations, design patterns, system comparisons, and real interview scenarios.

Section 1: Asynchronous Communication Fundamentals

Q1. What is the difference between synchronous and asynchronous communication?

In **synchronous communication**, the caller sends a request and blocks (waits) until the receiver processes it and returns a response — e.g., REST/gRPC calls. The caller's thread is tied up, and the caller's availability is coupled to the callee's availability.

In **asynchronous communication**, the caller sends a message and continues its work without waiting. The message is placed on an intermediary (queue/broker) and processed by the consumer whenever it is ready. The producer and consumer are decoupled in **time** (consumer can be down temporarily), **space** (they don't need to know each other's location), and **load** (the broker absorbs traffic spikes).

Example: Order placement returns 200 OK immediately after publishing an OrderCreated event; invoice generation, notification, and inventory updates happen asynchronously downstream.

Q2. What problems does asynchronous messaging solve in microservices?

1. **Temporal decoupling:** Consumer downtime doesn't fail the producer — messages wait in the broker.
2. **Load leveling:** Traffic spikes are buffered; consumers process at their own sustainable rate.
3. **Elastic scaling:** Add consumers to a group to increase throughput without touching producers.
4. **Resilience:** Failure in one downstream service doesn't cascade to the caller.
5. **Fan-out:** One event can drive many independent consumers (analytics, notifications, search indexing).
6. **Auditability/replay:** With a log-based broker like Kafka, events can be replayed to rebuild state or backfill new services.

Q3. What are the trade-offs / disadvantages of asynchronous communication?

1. **Eventual consistency:** Downstream state lags behind; you cannot read-your-own-writes across services immediately.
2. **Complex error handling:** No immediate response; failures must be handled via retries, DLQs, and monitoring.
3. **Debugging difficulty:** Flows span services and time; requires correlation IDs and distributed tracing.
4. **Duplicate messages:** Most systems give at-least-once delivery, so consumers must be idempotent.
5. **Ordering challenges:** Global ordering is expensive; usually only per-key/per-partition ordering is guaranteed.
6. **Operational overhead:** A broker cluster is another critical piece of infrastructure to run, monitor, and upgrade.

Q4. Message Queue vs Publish-Subscribe — what's the difference?

Message Queue (point-to-point): Each message is consumed by exactly one consumer. Consumers compete for messages. Once acknowledged, the message is removed. Good for task distribution (e.g., image-processing jobs). Examples: RabbitMQ queues, SQS.

Publish-Subscribe: Each message is delivered to all subscribers (or all subscriber groups). Good for event broadcasting where multiple services react to the same event.

Kafka unifies both: Within a consumer group, partitions are divided among members (queue semantics — competing consumers). Across different consumer groups, every group gets the full stream (pub-sub semantics). This is one of Kafka's most elegant design points.

Q5. What is a message broker vs an event streaming platform?

A **message broker** (RabbitMQ, ActiveMQ) routes and delivers messages, typically deleting them after acknowledgment. It's optimized for smart routing (exchanges, bindings, priorities) and transient work distribution.

An **event streaming platform** (Kafka, Pulsar) stores events in a durable, append-only, replayable log with configurable retention. Consumers track their own position (offset). This enables replay, multiple independent readers at different positions, stream processing, and event sourcing.

Rule of thumb: broker = 'deliver this task once and forget'; streaming platform = 'record what happened, let anyone read it, now or later'.

Q6. What is event-driven architecture (EDA)? What are its main patterns?

EDA is an architectural style where services communicate by producing and reacting to **events** (immutable facts: 'OrderPlaced', 'PaymentCaptured') rather than direct commands.

Patterns:
1. **Event notification:** Lightweight event says 'something changed'; consumers call back for details.
2. **Event-carried state transfer:** Event carries the full state, so consumers don't need to call back — better decoupling, larger payloads.
3. **Event sourcing:** The event log IS the source of truth; current state is derived by replaying events.
4. **CQRS:** Separate write model (commands) and read model (projections built from events).

Q7. Command vs Event vs Query — differentiate.

Command: An instruction to do something, addressed to one handler, can be rejected — 'ReserveInventory'. Imperative, usually named in present tense.
Event: An immutable fact about something that already happened — 'InventoryReserved'. Past tense, broadcast, cannot be rejected (it already occurred); consumers decide how to react.
Query: A request for data with no side effects.

In Kafka-based systems, commands often go on dedicated request topics with a single owning service, while events go on shared event topics with many subscribers.

Q8. What delivery guarantees exist in messaging systems?

1. **At-most-once:** Messages may be lost but never redelivered. Fastest; acceptable for metrics/telemetry where occasional loss is fine. (Fire-and-forget producer, or committing offsets before processing.)
2. **At-least-once:** Messages are never lost but may be redelivered. The default practical guarantee; requires **idempotent consumers**. (Commit offsets after processing.)
3. **Exactly-once:** Each message affects state exactly once. Extremely hard end-to-end; Kafka achieves it within its ecosystem via idempotent producers + transactions, but across external systems (DB, email) you approximate it with at-least-once + idempotency/dedup.

Q9. What is backpressure and how do you handle it in async systems?

Backpressure is the situation where producers generate messages faster than consumers can process, causing unbounded queue growth, memory exhaustion, or growing lag.

Handling strategies:
1. **Buffering:** The broker absorbs bursts (Kafka retention makes this natural).
2. **Consumer scaling:** Add consumers/partitions to raise processing throughput.
3. **Rate limiting / throttling** producers (quotas in Kafka).
4. **Load shedding:** Drop or sample low-priority messages.
5. **Pull-based consumption:** Kafka consumers pull at their own pace (max.poll.records), which is inherent backpressure — unlike push systems that can overwhelm consumers.
6. **Pause/resume:** Kafka consumer API lets you pause() partitions when internal buffers fill.

Q10. What is idempotency and why is it critical for async consumers?

An operation is idempotent if applying it multiple times has the same effect as applying it once. Because at-least-once delivery causes duplicates (producer retries, consumer rebalances, redelivery after crash before offset commit), consumers MUST handle re-processing safely.

Techniques:
1. **Natural idempotency:** 'SET status = SHIPPED' vs 'increment counter'.
2. **Dedup table:** Store processed message IDs (or business keys) with a unique constraint; skip if already present.
3. **Upserts** keyed on business ID.
4. **Conditional updates:** Version checks / optimistic locking ('update if version = N').
5. **Idempotency keys** passed to downstream APIs (e.g., payment gateways).

Q11. Explain correlation ID and its role in async systems.

A correlation ID is a unique identifier attached to a request at the entry point and propagated through every message, log line, and downstream call in that flow. In async systems where a single business transaction spans multiple topics and services over time, the correlation ID is the only practical way to stitch together logs and traces for debugging.

In Kafka it's typically carried in **message headers** and propagated automatically via tracing instrumentation (OpenTelemetry, Sleuth). It also enables the **request-reply over messaging** pattern: the reply message carries the same correlation ID so the requester can match responses to pending requests.

Q12. How do you implement request-reply over asynchronous messaging?

1. Requester publishes a message to a **request topic**, including a **correlation ID** and a **reply-to** topic (often in headers).
2. The responder processes it and publishes the result to the reply-to topic with the same correlation ID.
3. The requester consumes the reply topic and matches responses to pending requests (e.g., a map of correlationId → CompletableFuture, completing the future on arrival).

Spring Kafka provides **ReplyingKafkaTemplate** for exactly this. Caveats: adds latency vs direct RPC, needs timeouts for lost replies, and per-instance reply topics (or partition assignment) so replies reach the right requester instance.

Section 2: Kafka Fundamentals

Q13. What is Apache Kafka and what are its core use cases?

Kafka is a distributed, partitioned, replicated **commit log** used as an event streaming platform. Producers append records to topics; consumers read them at their own pace by offset.

Core use cases:
1. **Messaging / service decoupling** in microservices.
2. **Activity tracking:** clickstreams, user events at massive scale (its original LinkedIn use case).
3. **Log/metrics aggregation** pipelines.
4. **Stream processing:** real-time transformations with Kafka Streams/Flink.
5. **Event sourcing & CDC:** capturing DB changes via Debezium.
6. **Data integration backbone** between OLTP systems, caches, search indexes, and warehouses.

Q14. Explain Kafka's core components: topic, partition, offset, broker, producer, consumer.

Topic: A named, logical stream of records (e.g., 'orders').
Partition: A topic is split into partitions — ordered, immutable, append-only logs. Partitions are the unit of parallelism and distribution.
Offset: A monotonically increasing sequence number identifying each record's position within a partition. Consumers track progress by offset.
Broker: A Kafka server that stores partitions and serves produce/fetch requests. A cluster is a set of brokers.
Producer: Client that publishes records, choosing the target partition (by key hash, round-robin/sticky, or explicitly).
Consumer: Client that pulls records from partitions, usually as part of a consumer group.

Q15. Why does Kafka use partitions? What do they enable?

1. **Scalability:** A topic's data and traffic are spread across brokers; no single machine must hold the whole topic.
2. **Parallelism:** Each partition is consumed by at most one consumer within a group, so partition count caps consumer parallelism.
3. **Ordering:** Kafka guarantees order **within a partition** only. Keyed messages (same key → same partition) get per-key ordering — e.g., all events for order #123 arrive in order.
4. **Replication unit:** Each partition is replicated independently for fault tolerance.

Trade-off: More partitions = more parallelism but more open file handles, more replication traffic, longer leader elections, and more memory in clients.

Q16. What is a consumer group and how does partition assignment work?

A consumer group is a set of consumers sharing a group.id that cooperatively consume a topic: each partition is assigned to exactly one member of the group. This gives horizontal scaling (add members up to the partition count) and queue semantics within the group, while different groups independently read the full stream (pub-sub).

Assignment: One broker acts as the **group coordinator**. One consumer is elected **group leader** and runs the assignment strategy: Range, RoundRobin, Sticky, or **CooperativeSticky** (incremental rebalancing, now the recommended default). If members join/leave/fail (missed heartbeats), a **rebalance** redistributes partitions.

Q17. What happens if there are more consumers than partitions?

Extra consumers sit **idle** — a partition can be assigned to only one consumer per group, so with 4 partitions and 6 consumers, 2 consumers get nothing. They do act as hot standbys: if an active consumer dies, a rebalance hands its partitions to an idle one.

This is why partition count should be planned with target consumer parallelism in mind, and why 'just add more consumers' stops working at the partition ceiling — at that point you must add partitions (carefully, since it changes key→partition mapping for keyed data) or make each consumer faster.

Q18. How do consumers track their position? Explain offset commits.

Each consumer tracks the offset of the next record to read per partition. Committed offsets are stored in the internal compacted topic **__consumer_offsets**, keyed by (group, topic, partition).

Auto-commit (enable.auto.commit=true): offsets are committed every auto.commit.interval.ms (default 5s) during poll(). Risky — a crash after commit but before finishing processing loses messages; a crash after processing but before commit duplicates them.
Manual commit: commitSync() (blocking, retries, safest), commitAsync() (non-blocking, no retry, use with callback), or committing specific offsets per partition for fine-grained control. The standard at-least-once pattern: process the batch, then commitSync.

Q19. What is consumer lag and how do you monitor and fix it?

Lag = (log end offset) – (committed consumer offset), per partition — how far behind the consumer is.

Monitoring: kafka-consumer-groups.sh --describe, Burrow, Prometheus/Grafana with kafka_consumergroup_lag, Confluent Control Center.

Fixes:
1. Scale consumers (up to partition count) or add partitions.
2. Speed up processing: batch DB writes, async/parallel processing per batch, remove blocking I/O from the poll loop.
3. Tune fetch: max.poll.records, fetch.min.bytes, fetch.max.wait.ms.
4. Move heavy work out: consume fast, hand off to a worker pool or another topic.
5. Check for rebalance storms or a poison-pill message stalling one partition — lag skewed on one partition usually means a hot key or a stuck message.

Q20. Explain Kafka message structure: key, value, headers, timestamp.

Key (optional): Bytes used for partitioning (murmur2 hash of key % partitions) and for log compaction identity. Same key → same partition → ordered per key.
Value: The payload (JSON, Avro, Protobuf...). Can be null — a null value with a key is a **tombstone** that deletes the key in compacted topics.
Headers: Key-value metadata pairs — correlation IDs, trace context, schema info, event type — letting consumers route/filter without deserializing the payload.
Timestamp: CreateTime (producer-set) or LogAppendTime (broker-set), used for retention, time-based offset lookup, and stream-processing windowing.

Q21. What is Kafka retention? Time vs size vs compaction.

Kafka retains records regardless of consumption, per topic policy:
1. **Time-based:** retention.ms (default 7 days) — segments older than this are deleted.
2. **Size-based:** retention.bytes per partition — oldest segments deleted when exceeded.
3. **Log compaction** (cleanup.policy=compact): keeps at least the latest value for every key, deleting older values for the same key. Ideal for changelog/state topics (latest account balance, latest config). Tombstones (null values) remove keys entirely after delete.retention.ms.

Policies can combine: 'compact,delete'. Retention is why consumers can replay — a new service can bootstrap by reading from earliest.

Q22. How does a producer decide which partition a message goes to?

1. **Explicit partition** specified in the ProducerRecord → used directly.
2. **Key present:** partition = murmur2(keyBytes) % numPartitions. Deterministic, so a key always lands on the same partition (until partition count changes — which is why adding partitions breaks key ordering guarantees for existing keys).
3. **No key:** Modern clients use the **sticky partitioner** — fill a batch to one partition, then switch — which improves batching/latency vs old round-robin per message.
4. **Custom Partitioner** implementation for special routing (e.g., a VIP tenant pinned to dedicated partitions).

Q23. What are serializers and deserializers in Kafka?

Kafka brokers only store bytes; clients convert objects ↔ bytes. Producers configure key.serializer / value.serializer; consumers configure matching deserializers. Common: StringSerializer, ByteArraySerializer, JsonSerializer, and schema-registry-aware Avro/Protobuf serializers.

Pitfalls: mismatched serializer/deserializer causes SerializationException — which can become a **poison pill** that crashes the consumer loop repeatedly. Handle with ErrorHandlingDeserializer (Spring Kafka) or try/catch + DLQ so one bad record doesn't stall the partition.

Q24. What is the difference between kafka's pull model and a push model? Why did Kafka choose pull?

Kafka consumers **pull** (poll) records from brokers rather than brokers pushing to consumers.

Why pull:
1. **Natural backpressure:** Consumers fetch only what they can handle — a push broker must guess consumer capacity and risks overwhelming slow consumers.
2. **Aggressive batching:** A consumer can pull large contiguous chunks efficiently; push systems tend to send message-by-message or need complex accumulation logic.
3. **Replay & position control:** Consumers own their offset and can rewind/skip — trivial with pull, awkward with push.

To avoid busy-polling when there's no data, fetch requests support long polling: the broker parks the request until fetch.min.bytes accumulate or fetch.max.wait.ms elapses.

Section 3: Kafka Architecture & Internals

Q25. Describe Kafka's replication model: leader, followers, ISR.

Each partition has **replication-factor** copies across brokers. One replica is the **leader** — all produces and (by default) fetches go through it. Others are **followers**, which replicate by fetching from the leader like consumers.

ISR (In-Sync Replicas): The set of replicas fully caught up with the leader (within replica.lag.time.max.ms, default 30s). A message is **committed** when all ISR members have it; only committed messages are visible to consumers (the high watermark). If the leader dies, a new leader is elected from the ISR, guaranteeing no committed data loss (with proper settings).

Q26. Explain acks=0, acks=1, acks=all and min.insync.replicas.

acks=0: Producer doesn't wait for any acknowledgment. Highest throughput, messages lost silently if the broker fails. Fire-and-forget telemetry only.
acks=1: Leader acknowledges after writing locally. If the leader dies before followers replicate, acknowledged data is lost.
acks=all (-1): Leader acknowledges only after all ISR replicas have the record.

min.insync.replicas (broker/topic config): The minimum ISR size for acks=all writes to succeed. With RF=3 and min.insync.replicas=2, the cluster tolerates one broker down while still guaranteeing every acked write exists on ≥2 brokers. If ISR shrinks below it, producers get NotEnoughReplicasException — choosing consistency over availability. The classic durable setup: **RF=3, min.insync.replicas=2, acks=all**.

Q27. What is unclean leader election? When would you enable it?

If all ISR replicas of a partition die, Kafka must choose: wait for an ISR member to return (unavailable but consistent) or elect an out-of-sync replica as leader (available but loses the messages that replica missed).

unclean.leader.election.enable=false (default) chooses consistency — the partition stays offline until an in-sync replica recovers. Setting it true chooses availability at the cost of possible data loss and is only defensible for streams where staying up matters more than completeness (e.g., metrics, clickstream sampling) — never for orders, payments, or ledgers.

Q28. What roles did ZooKeeper play, and what is KRaft?

ZooKeeper historically stored cluster metadata: broker registration, topic configs, partition leadership, ACLs, and elected the cluster **controller**. Problems: a second distributed system to operate, metadata scalability ceiling (~200k partitions), slow controller failover on large clusters.

KRaft (Kafka Raft) replaces ZooKeeper: a quorum of controller nodes runs the Raft consensus protocol over an internal metadata log (__cluster_metadata). Benefits: single system to operate, millions of partitions, near-instant controller failover (standby controllers already have the metadata replicated), and metadata itself follows Kafka's log paradigm. KRaft is production-ready since Kafka 3.3, and Kafka 4.0 removes ZooKeeper entirely.

Q29. What is the controller in a Kafka cluster?

The controller is the broker (or KRaft quorum leader) responsible for cluster-wide administrative decisions: detecting broker failures, electing partition leaders, reassigning partitions, and propagating metadata changes to all brokers. In ZooKeeper mode, one broker won an ephemeral-node race to become controller; failover required rebuilding state from ZooKeeper (slow on big clusters). In KRaft, the active controller is the Raft leader of the controller quorum, and standbys have hot replicated state — failover is nearly instantaneous.

Q30. Explain Kafka's storage internals: segments, indexes, and why sequential I/O matters.

Each partition is stored as a directory of **segments** — fixed-size log files (segment.bytes, default 1GB) plus index files: an **offset index** (offset → file position) and a **time index** (timestamp → offset). Only the **active segment** is written; older segments are immutable and eligible for retention/compaction.

Performance comes from:
1. **Sequential appends** — writes go to the end of the active segment; sequential disk I/O approaches RAM-like throughput vs random I/O.
2. **OS page cache** — Kafka doesn't maintain its own data cache; reads of recent data are served from page cache.
3. **Zero-copy (sendfile):** Data goes page cache → NIC without copying through user space (for non-TLS, uncompressed-passthrough paths).
4. **Binary format parity:** Producer, broker, and consumer share the same record-batch format, so brokers pass compressed batches through without recompression.

Q31. What is the high watermark and log end offset (LEO)?

LEO (Log End Offset): The offset of the next record to be appended in a replica — each replica has its own LEO. **High Watermark (HW):** The minimum LEO across the ISR — the highest offset known replicated to all in-sync replicas. Consumers can read only up to the HW; records above it are not yet 'committed' and could be lost on leader failover, so exposing them would risk reading data that later disappears.

Follower fetch responses carry the HW so followers can advance theirs; this lag in HW propagation is why leader changes can briefly cause consumers to see non-monotonic behavior handled by fetch sessions/epochs.

Q32. What is a leader epoch and why was it introduced?

A leader epoch is a monotonically increasing number incremented on every leader election for a partition, stored with the records. Before epochs, followers truncated their logs based on the high watermark alone after a leader change, which had edge cases causing data loss or divergence (KIP-101). With epochs, a recovering replica asks the new leader 'what's the end offset for epoch X?' and truncates precisely to the point where the logs diverge — guaranteeing replicas converge to identical logs without losing committed data.

Q33. How does Kafka handle very large messages? What are the best practices?

Kafka defaults cap messages around 1MB (message.max.bytes broker-side, max.request.size producer-side, fetch.max.bytes consumer-side). Raising limits works but hurts broker memory, page cache efficiency, and rebalance/replication behavior.

Best practices:
1. **Claim-check pattern:** Store the blob in object storage (S3), publish a small event with the reference/URL.
2. **Compression** (lz4/zstd) at the producer for compressible payloads.
3. **Chunking:** Split payloads and reassemble by key — complex, rarely worth it.
4. Keep events as facts/references, not file transfer — Kafka is a log, not a blob store.

Q34. Explain Kafka networking: how does a client find partition leaders?

1. Client connects to any broker from **bootstrap.servers**.
2. It sends a **Metadata request**; the broker returns the cluster map: brokers, topics, partitions, and each partition's current leader.
3. The client connects directly to leaders and routes produce/fetch requests to the right broker — there is no proxy layer; clients are cluster-aware.
4. On NotLeaderForPartition errors (leadership moved), the client refreshes metadata and re-routes.
advertised.listeners must expose addresses reachable by clients — the #1 cause of 'works on the broker host, fails remotely' issues, especially in Docker/K8s.

Q35. What are Kafka quotas and why do they matter in multi-tenant clusters?

Quotas throttle clients to protect the cluster from noisy neighbors: **produce/fetch byte-rate quotas** (per client.id, user, or both) and **request-rate quotas** (percentage of broker request-handler time). When exceeded, the broker delays responses (throttling) rather than erroring, so clients transparently slow down.

In shared clusters (e.g., an org-wide event bus), quotas prevent one team's backfill job from starving latency-sensitive production consumers, and protect brokers from metadata-request storms from misconfigured clients.

Q36. Describe what happens end-to-end when a producer sends a message.

1. **send()** serializes key/value, runs interceptors, and the partitioner picks a partition.
2. Record is appended to an in-memory **RecordAccumulator** batch for that partition (batch.size, linger.ms control batching).
3. The **sender thread** drains ready batches, groups them by leader broker, and sends ProduceRequests (up to max.in.flight.requests.per.connection outstanding).
4. The leader validates, appends to the active segment, and (acks=all) waits for ISR followers to fetch and replicate.
5. Leader responds; producer completes the future/callback. On retriable errors it retries (retries, delivery.timeout.ms) — with idempotence enabled, sequence numbers prevent duplicates from retries.
6. Consumers see the record once the high watermark passes its offset.

Section 4: Producer Deep Dive

Q37. Explain producer batching: batch.size and linger.ms.

The producer groups records per partition into batches to amortize network and I/O costs.

batch.size (default 16KB): max bytes per batch — a batch is sent when full.
linger.ms (default 0): how long to wait for a batch to fill before sending anyway. linger.ms=0 sends immediately (lowest latency, poor batching under light load); linger.ms=5–20 trades a few ms of latency for dramatically better throughput and compression ratios.

A batch is dispatched when EITHER it is full OR linger expires. Under heavy load, batches fill instantly and linger barely matters; under light load, linger is what creates batching. Also relevant: buffer.memory (total producer buffer, default 32MB) — when full, send() blocks up to max.block.ms.

Q38. What is the idempotent producer and how does it work internally?

enable.idempotence=true (default since Kafka 3.0) guarantees that producer retries never create duplicates **within a partition**.

Mechanism: The producer gets a unique **Producer ID (PID)** from the broker and attaches a monotonically increasing **sequence number** per partition to every batch. The broker tracks the last sequence per (PID, partition): a duplicate sequence (retry of an already-written batch) is acknowledged but not re-written; an out-of-order gap raises OutOfOrderSequenceException.

It enforces acks=all, retries > 0, and max.in.flight ≤ 5 (broker can deduplicate/reorder within 5 in-flight batches). Scope: single producer session, single partition — it does NOT dedupe across producer restarts (PID changes) or across app-level re-sends; that requires transactions or consumer-side idempotency.

Q39. Does retries + max.in.flight.requests > 1 break ordering?

Without idempotence: yes. If batch A fails and batch B (sent after A on the same partition) succeeds, the retry of A lands after B — reordering. The old workaround was max.in.flight.requests.per.connection=1, which serializes requests and hurts throughput.

With **idempotence enabled**, sequence numbers let the broker detect and reject out-of-order batches, so ordering is preserved with up to 5 in-flight requests — you get pipelining AND ordering. This is why idempotence became the default: it removes the classic ordering-vs-throughput dilemma.

Q40. Compare compression codecs in Kafka: gzip, snappy, lz4, zstd.

compression.type is set on the producer (and optionally topic). Compression applies per batch — bigger batches compress better (another reason to tune linger.ms).

gzip: Best ratio, highest CPU, slowest. Cross-DC links where bandwidth is precious.
snappy: Fast, moderate ratio — the long-time balanced default choice.
lz4: Fastest decompression, good ratio — great for high-throughput, latency-sensitive paths.
zstd (Kafka 2.1+): Near-gzip ratios at near-lz4 speeds; the modern recommendation for most workloads.

Brokers store batches compressed and pass them through to consumers (no recompression if topic config matches), so compression saves network AND disk.

Q41. What is delivery.timeout.ms and how does it relate to retries and request.timeout.ms?

delivery.timeout.ms (default 120s) is the total time budget for a send: batching wait + in-flight time + all retries. When it expires, the send fails permanently and the callback gets the last error.
request.timeout.ms (default 30s): how long to wait for a single request's response before considering it failed (and retrying).
retries (default MAX_INT): retry count — but in modern clients, delivery.timeout.ms is the real limiter; you reason in time, not counts.

Constraint: delivery.timeout.ms ≥ linger.ms + request.timeout.ms. Design view: pick delivery.timeout as 'how long can my app tolerate not knowing the fate of this message', and handle the final failure (log, fallback store, alert) in the callback.

Q42. How do you handle producer failures in a critical flow (e.g., order events)?

1. **Config for durability:** acks=all, enable.idempotence=true, RF=3 + min.insync.replicas=2.
2. **Handle the callback:** Never fire-and-forget for critical data — on failure after retries, persist to a fallback (DB table / local disk) and alert, or fail the originating operation.
3. **Transactional outbox:** Write the event to an outbox table in the same DB transaction as the business change; a relay (or Debezium CDC) publishes to Kafka. This removes the dual-write problem entirely — the DB commit is the source of truth.
4. **Monitoring:** record-error-rate, buffer-exhausted-rate, request-latency metrics with alerts.
5. Beware max.block.ms: if the broker is down and buffers fill, send() blocks — decide whether the calling thread may block or should shed load.

Q43. What is the dual-write problem? Explain the transactional outbox pattern.

Dual-write problem: A service must update its database AND publish an event. Two separate systems cannot be updated atomically — if the DB commit succeeds but the Kafka publish fails (or vice versa, or the process crashes between them), state and events diverge: downstream services never learn about an order, or learn about one that was rolled back.

Transactional outbox:
1. In ONE local DB transaction, write the business change AND insert the event into an **outbox table**. Atomic by ACID.
2. A separate publisher reads the outbox and produces to Kafka: either a **polling relay** (SELECT unpublished → send → mark published) or **CDC (Debezium)** tailing the DB log — CDC avoids polling latency/load.
3. Publishing is at-least-once (relay may crash after send, before marking) → consumers must be idempotent, typically keyed by the outbox event ID.

Result: an event is published if and only if the transaction committed.

Q44. What are producer interceptors and when would you use them?

ProducerInterceptor (and ConsumerInterceptor) hook into the client pipeline: onSend() can inspect/modify records before serialization; onAcknowledgement() observes results. Use cases: injecting tracing headers (OpenTelemetry instrumentation works this way), audit logging, metrics enrichment, tenant tagging, lightweight PII redaction. They should be fast and side-effect-safe — exceptions are swallowed/logged, and heavy work here adds latency to every send.

Q45. How would you guarantee strict ordering for events of one entity while keeping throughput high?

1. **Key by entity ID** (orderId, userId): all events for an entity go to one partition, preserving their relative order, while different entities spread across partitions for parallelism.
2. Enable **idempotence** so retries can't reorder.
3. Consumer side: process records of a partition **in order per key** — if you parallelize a batch across threads, shard the work by key (e.g., hash key → worker thread) so one entity's events never interleave.
4. Avoid increasing partition count later on keyed topics (remapping breaks 'same key, same partition' across the boundary) — over-provision partitions upfront or plan a topic migration.
5. Watch for **hot keys**: one huge entity can saturate a single partition — mitigations include sub-keying (orderId + shardHint) when strict total order per entity isn't truly required.

Section 5: Consumer Deep Dive & Rebalancing

Q46. Explain the consumer poll loop and max.poll.interval.ms vs session.timeout.ms.

The consumer is single-threaded around **poll()**: poll fetches records, drives group heartbeating coordination, and signals liveness.

session.timeout.ms (default 45s): liveness via **heartbeats** sent by a background thread every heartbeat.interval.ms (3s). If the coordinator misses heartbeats for the session timeout, the consumer is declared dead → rebalance. Detects crashed/hung processes.
max.poll.interval.ms (default 5 min): max allowed gap between poll() calls. If processing a batch takes longer, the consumer is ejected from the group even though heartbeats continue — detecting livelock ('alive but stuck processing').

Classic failure: heavy per-record work × max.poll.records exceeds max.poll.interval → endless rebalance loop. Fixes: fewer records per poll, faster processing, async hand-off, or raising the interval deliberately.

Q47. What triggers a consumer group rebalance, and what happens during one?

Triggers: member joins (deploy, scale-up), member leaves/crashes (missed heartbeats, exceeded max.poll.interval), topic metadata change (partitions added, regex subscription matches new topic), or coordinator change.

Eager (classic) protocol: ALL members revoke ALL partitions, rejoin, the leader computes a full assignment, everyone resumes — a stop-the-world pause for the whole group.
Cooperative (incremental) protocol (CooperativeStickyAssignor): members keep partitions not being moved; only reassigned partitions are revoked in a second sync round. Consumption continues on unaffected partitions — dramatically better for large groups and rolling deploys.

During rebalance, in-flight processing of revoked partitions must complete/commit in the onPartitionsRevoked callback or those records will be reprocessed by the new owner (at-least-once duplicates).

Q48. What is static group membership and what problem does it solve?

Setting **group.instance.id** gives a consumer a stable identity. Normally, any restart makes the consumer look like a new member (new member ID) → leave + join → two rebalances per rolling restart per instance.

With static membership, a bounce within session.timeout.ms lets the returning instance reclaim its exact partitions with **no rebalance at all**. Combined with a raised session timeout (e.g., 2–5 min) and CooperativeSticky assignment, Kubernetes rolling deploys of large consumer groups go from rebalance storms to near-zero disruption. Trade-off: a genuinely dead static member isn't replaced until session.timeout expires, delaying failover.

Q49. Compare partition.assignment.strategy options: Range, RoundRobin, Sticky, CooperativeSticky.

Range (old default): Per topic, sorts partitions and members, assigns contiguous chunks. Skews load when consuming many topics (member #1 gets partition 0 of EVERY topic) — but co-partitions same-numbered partitions across topics, useful for joins.
RoundRobin: Distributes all partitions across all members evenly, ignoring previous assignment — every rebalance can reshuffle everything (bad for local state/caches).
Sticky: Even distribution AND minimal movement vs previous assignment — but still uses the eager protocol (full revoke).
CooperativeSticky (recommended): Sticky assignment + incremental cooperative protocol — minimal movement and no stop-the-world. The default in Spring Kafka 3.x era setups and Kafka Streams' internal assignor is similarly cooperative.

Q50. How do you achieve at-least-once, at-most-once, and (effectively) exactly-once as a consumer?

At-least-once: Disable auto-commit (or use containers that commit after listener success); process, THEN commit. Crash before commit → reprocess → duplicates possible → make handlers idempotent.
At-most-once: Commit BEFORE processing (or auto-commit with fast polls). Crash after commit, before processing → message lost. Rarely what you want.
Effectively exactly-once (with external systems): at-least-once + idempotency: dedup table keyed on message/business ID, upserts, or conditional writes. Alternative: store the consumed offset IN the same DB transaction as the state change, and on startup seek() to the DB-stored offset — offset and effect commit atomically.
True EOS within Kafka: read-process-write via transactions (next section).

Q51. What is a poison pill message and how do you handle it?

A poison pill is a record that deterministically fails processing (corrupt bytes, deserialization failure, business rule crash). Since Kafka doesn't remove records on failure, the consumer retries the same record forever, stalling the partition — lag grows while every other partition proceeds.

Handling:
1. **Deserialization safety:** ErrorHandlingDeserializer (Spring) or byte-array deserializers with manual parsing in try/catch.
2. **Bounded retries + DLQ:** After N attempts (with backoff), publish the failing record + error metadata to a dead-letter topic and advance the offset. Spring Kafka: DefaultErrorHandler + DeadLetterPublishingRecoverer.
3. **DLQ workflow:** Alerting on DLQ depth, tooling to inspect, fix, and replay from DLQ back to the source topic.
4. Distinguish **transient** errors (DB timeout → retry, maybe pause partition) from **permanent** ones (bad schema → DLQ immediately).

Q52. Explain retry strategies for consumers: blocking retry, retry topics, and backoff.

Blocking (in-place) retry: Retry the record in the poll loop with backoff. Simple and preserves order, but blocks the whole partition and risks exceeding max.poll.interval.ms — keep total retry time bounded.

Non-blocking retry topics (tiered retries): On failure, publish to retry topic(s) — e.g., orders-retry-5s, orders-retry-1m, orders-retry-10m — each consumed after a delay; final failure goes to the DLQ. Main partition keeps flowing. Trade-off: **per-key ordering is lost** (a failed record's successor may process first), so it suits independent events, not strict per-entity sequences. Spring Kafka's @RetryableTopic automates this.

Backoff: Always exponential with jitter for transient downstream failures, to avoid retry storms hammering a recovering dependency.

Q53. How do you seek to a specific offset or timestamp? When is this needed?

APIs: seek(partition, offset), seekToBeginning(), seekToEnd(), and offsetsForTimes(timestamp) → then seek to the returned offsets.

Use cases:
1. **Replay after a bug:** Reset the group to the timestamp before a bad deploy and reprocess (kafka-consumer-groups.sh --reset-offsets --to-datetime ... also does this offline).
2. **Skipping a poison pill** surgically (seek past one offset).
3. **Bootstrapping** a new projection from earliest.
4. **DB-driven offsets** for atomic offset+state storage.

Note auto.offset.reset (earliest/latest/none) only applies when the group has NO committed offset (new group) or the committed offset was deleted by retention — it's not a general rewind mechanism.

Q54. How do you parallelize processing beyond the partition count?

Options when partitions are the ceiling but per-record work is slow:
1. **Worker-pool inside the consumer:** poll() a batch, dispatch records to an executor **sharded by key** (to keep per-key order), pause() the partition if the pool backs up, commit only offsets whose predecessors are all done (offset tracking becomes the hard part).
2. **Two-stage topology:** Consumer 1 does cheap validation and re-produces to a second topic with MORE partitions; consumer group 2 scales wider.
3. **Confluent Parallel Consumer** library: key-level or record-level parallelism with correct offset management out of the box.
4. Best fix when possible: **make processing async/batched** (bulk DB writes) rather than adding moving parts.

Q55. What is fetch.min.bytes / fetch.max.wait.ms and how do they affect latency vs throughput?

They control broker-side batching for consumer fetches:
fetch.min.bytes (default 1): The broker holds the fetch response until at least this many bytes are available.
fetch.max.wait.ms (default 500): ...or until this timeout, whichever first.

Raising fetch.min.bytes (e.g., 64KB) reduces request rate and CPU on both sides — great for high-throughput pipelines — at the cost of up to fetch.max.wait.ms extra latency under light traffic. Latency-critical consumers keep defaults. Related sizing: max.partition.fetch.bytes (per-partition cap) and fetch.max.bytes (per-response cap) must exceed the largest message or the consumer stalls.

Section 6: Exactly-Once, Transactions & Delivery Semantics

Q56. Explain Kafka transactions: what do they guarantee and how do they work?

Transactions make a set of writes to multiple partitions — plus the consumer offset commit — **atomic**: consumers with isolation.level=read_committed see all of them or none.

Mechanics:
1. Producer sets a stable **transactional.id**; initTransactions() registers with the **transaction coordinator** (a broker module) and bumps the **producer epoch** — fencing out zombie instances with the same ID (their writes are rejected).
2. beginTransaction() → sends to various partitions → sendOffsetsToTransaction(offsets, groupMetadata) ties the input offsets into the same transaction.
3. commitTransaction(): coordinator writes PREPARE to the internal __transaction_state topic, then writes **commit markers** into every touched partition, then COMPLETE. Abort works the same with abort markers.
4. read_committed consumers buffer records past the **Last Stable Offset** until markers resolve them; aborted records are filtered out.

Q57. What is the zombie fencing problem and how does transactional.id solve it?

A 'zombie' is an old producer instance presumed dead (GC pause, network partition) that resumes and writes after a replacement instance has taken over — causing duplicates or interleaved partial transactions.

With a stable transactional.id, every initTransactions() call increments the **epoch** for that ID at the coordinator. Any request from a producer with an older epoch is rejected with ProducerFenced. So when the new instance initializes, the zombie is fenced the moment it tries to write or commit. This is why transactional.id must be stable per logical producer identity (e.g., tied to the input partition assignment in EOS stream processing, which Kafka Streams manages automatically).

Q58. What does exactly-once semantics (EOS) actually cover in Kafka — and what doesn't it?

Covered: The **consume-process-produce loop within Kafka**: read from topic A, transform, write to topic B, committing input offsets and output records atomically. Kafka Streams turns this on with processing.guarantee=exactly_once_v2.

NOT covered: Side effects on external systems. If your consumer writes to Postgres, calls a payment API, or sends an email, Kafka transactions do not make those atomic — a retried transaction re-executes the external call.

For external systems: (a) idempotent writes keyed by event ID (upsert/dedup table), (b) store offsets in the external DB in the same local transaction as the state change, or (c) accept at-least-once + dedup at the boundary. Interviews love this distinction: 'EOS' is within the Kafka ecosystem; end-to-end exactly-once with arbitrary systems is achieved via idempotency, not magic.

Q59. What is isolation.level=read_committed and what is the Last Stable Offset (LSO)?

read_uncommitted (default): consumers see all records, including those from open or aborted transactions. read_committed: consumers only receive records from committed transactions (aborted ones are filtered client-side using abort markers) and can read only up to the **LSO** — the offset before the earliest still-open transaction in the partition.

Consequence: one long-running open transaction **stalls all read_committed consumers** on that partition, even for unrelated committed data after it. Hence transactions should be short and transaction.timeout.ms bounds runaway transactions (the coordinator aborts them). Lag monitoring for read_committed groups should account for LSO, not just LEO.

Q60. Compare exactly_once (v1) vs exactly_once_v2 in Kafka Streams.

EOS v1: One transactional producer **per input partition per instance** — clusters with many partitions suffered huge producer counts, coordinator load, and slow rebalances.
EOS v2 (KIP-447, Kafka 2.5+): One transactional producer **per stream thread**; fencing shifted from per-partition transactional IDs to **consumer group epochs** validated at offset-commit time. Massively fewer producers, faster rebalances, lower overhead — the reason EOS became practical at scale. Config: processing.guarantee=exactly_once_v2. Cost vs at_least_once remains: commit-interval-bounded latency (transactions commit every commit.interval.ms, default 100ms under EOS) and some throughput overhead.

Q61. Design: payments must be processed exactly once, but the processor calls an external bank API. What do you do?

You cannot wrap the bank call in a Kafka transaction, so:
1. **Idempotency key:** Derive a stable key per payment (paymentId) and pass it to the bank API — serious payment APIs support idempotency keys, making retries safe on their side.
2. **State machine in DB:** Before calling, upsert payment row to IN_PROGRESS (unique on paymentId); after success, mark COMPLETED with the bank reference. On redelivery: COMPLETED → skip; IN_PROGRESS → query the bank by idempotency key to resolve the ambiguous state, then finalize.
3. **At-least-once consumption** (commit after DB finalization), DLQ for permanent failures, alerting on stuck IN_PROGRESS rows.
4. Never at-most-once 'to avoid duplicates' — losing a payment is worse than handling a duplicate you're already protected against.

Section 7: Kafka Ecosystem — Streams, Connect, Schema Registry

Q62. What is Kafka Streams? How does it differ from a plain consumer?

Kafka Streams is a Java library (not a cluster) for stateful stream processing on Kafka topics: filter/map/join/aggregate/window with a high-level DSL (KStream/KTable) or low-level Processor API.

Beyond a plain consumer it provides:
1. **Managed local state** (RocksDB stores) backed by **changelog topics** for fault tolerance — state rebuilds on another instance after failure.
2. **Windowing & time semantics:** event-time processing, tumbling/hopping/sliding/session windows, grace periods for late data.
3. **Joins:** stream-stream (windowed), stream-table, table-table — with co-partitioning requirements.
4. **EOS** via exactly_once_v2.
5. **Elastic scaling:** tasks (one per input partition) redistribute across instances automatically; standby replicas cut failover time.

Alternatives: Flink (separate cluster, richer semantics, huge state), ksqlDB (SQL over Streams).

Q63. Explain KStream vs KTable vs GlobalKTable.

KStream: Each record is an independent event/fact — 'OrderPlaced(o1, 500)'. Aggregations consume all records.
KTable: A changelog view — each record is an **upsert** for its key; only the latest value per key matters, like a table materialized from a compacted topic. 'Current status of order o1'.
GlobalKTable: A KTable fully replicated to **every** instance (each reads ALL partitions). Enables joins without co-partitioning and on foreign keys, ideal for small reference/dimension data (product catalog, config). Cost: full copy per instance, bootstrap time, more load.

Duality: a stream is a table's changelog; a table is the latest-state fold of a stream — this stream-table duality underpins the whole DSL.

Q64. What is Kafka Connect and when do you use it instead of writing a custom producer/consumer?

Kafka Connect is a framework for streaming data between Kafka and external systems using off-the-shelf **connectors**: **source connectors** (DB/CDC → Kafka: Debezium, JDBC) and **sink connectors** (Kafka → S3, Elasticsearch, JDBC, Snowflake...).

Why Connect over custom code: declarative JSON config instead of code; distributed workers with fault tolerance and automatic task rebalancing; offset tracking built in; Single Message Transforms (SMTs) for light in-flight tweaks; dead-letter-queue support; a huge connector ecosystem.

Write custom clients when logic is genuinely application-specific (business processing), not for plumbing data in/out of standard systems.

Q65. What is CDC and how does Debezium work?

CDC (Change Data Capture) captures row-level database changes as events. **Debezium** is a set of Kafka Connect source connectors that tail the database's transaction log — MySQL binlog, Postgres WAL (logical replication), MongoDB oplog — rather than polling tables.

Properties: low latency, no missed changes (including deletes), no query load on the DB, events carry before/after images, and an initial **snapshot** phase bootstraps existing data. Ordering per key follows the DB commit order.

Use cases: transactional outbox publishing (Debezium's outbox event router), cache/search-index synchronization, replicating OLTP data to analytics, and strangler-pattern migrations off monolith databases.

Q66. Why do you need a Schema Registry? How does schema evolution work?

Kafka stores bytes; nothing broker-side stops a producer deploy from publishing a payload that breaks every consumer. A **Schema Registry** centrally stores versioned schemas (Avro/Protobuf/JSON Schema) and enforces **compatibility rules** at registration time. Serialized messages carry a small schema ID (magic byte + 4-byte ID) instead of the full schema.

Compatibility modes:
BACKWARD (default): new schema can read old data — consumers upgrade first; you may delete fields / add optional (defaulted) fields.
FORWARD: old schema can read new data — producers upgrade first; add fields / delete optional ones.
FULL: both — only add/remove fields that have defaults.

Rules of thumb: never rename or change types of existing fields; always give new fields defaults; treat schemas as APIs with review; use *_TRANSITIVE modes so checks run against all prior versions, not just the latest.

Q67. Avro vs Protobuf vs JSON for Kafka payloads — how do you choose?

JSON (plain): Human-readable, zero tooling, but large, slow to parse, no enforced schema — schema drift becomes runtime errors. OK for low-volume internal topics and early prototypes.
Avro: Compact binary, rich schema evolution semantics, first-class Schema Registry & Connect/Debezium integration, dynamic typing (schema travels via registry). The data-engineering ecosystem default.
Protobuf: Compact, fast, strong typing with codegen across many languages, natural if the org already uses gRPC — evolution rules are field-number-based.

Choose Avro for data pipelines/CDC-heavy stacks, Protobuf for polyglot microservice orgs standardized on proto contracts; either beats JSON at scale on size, speed, and safety.

Q68. What is MirrorMaker 2 / cluster replication used for?

MirrorMaker 2 (built on Kafka Connect) replicates topics, configurations, ACLs, and **consumer group offsets** (with offset translation) between clusters. Use cases:
1. **Disaster recovery:** active-passive clusters across regions; on failover, consumers resume from translated offsets.
2. **Geo-locality / aggregation:** regional clusters mirrored into a central analytics cluster.
3. **Migration** between clusters/providers with dual-run cutover.

Design notes: remote topics are prefixed (source.topic) to avoid cycles in active-active; replication is asynchronous, so RPO > 0 — cross-cluster 'exactly-once' is not provided. Alternatives: Confluent Cluster Linking (offset-preserving, no Connect), or stretch clusters for synchronous multi-AZ.

Section 8: Operations, Performance & Tuning

Q69. How do you decide the number of partitions for a topic?

Work from throughput and parallelism targets:
1. **Throughput math:** partitions ≥ max(target throughput ÷ per-partition producer throughput, target ÷ per-consumer throughput). If consumers each handle 10MB/s and you need 100MB/s → ≥10 partitions.
2. **Consumer parallelism ceiling:** partitions = max useful consumers per group; leave headroom (e.g., 2–3× current need) because increasing later breaks key→partition mapping.
3. **Key cardinality & skew:** enough partitions that hot keys don't dominate; but partitions ■ keys wastes overhead.
4. **Cluster limits:** each partition costs file handles, replication fetchers, memory in clients, and longer failovers — keep per-broker partition counts within recommended bounds (thousands, not hundreds of thousands, per broker; KRaft raised cluster ceilings, not broker resource costs).
5. Round to a highly divisible number (12, 24, 30) so consumer counts divide evenly.

Q70. What are the most important producer/consumer/broker configs for throughput tuning?

Producer: linger.ms 5–20, batch.size 64–256KB, compression.type=lz4/zstd, buffer.memory sized for bursts, acks per durability need.
Consumer: fetch.min.bytes up (e.g., 64KB), max.poll.records tuned to processing speed, max.partition.fetch.bytes for big batches, and efficient processing (bulk writes downstream).
Broker: num.io.threads / num.network.threads sized to cores, num.replica.fetchers up if replication lags, socket buffer sizes for cross-DC, log.segment.bytes default is fine; ensure sufficient page cache (RAM) — Kafka's read path lives on it; use multiple data dirs / fast disks (or tiered storage for cold data).
OS: file descriptor limits, vm.swappiness=1, XFS/ext4, noatime.

Q71. A consumer group's lag is growing steadily. Walk through your diagnosis.

1. **Scope it:** All partitions or a few? (kafka-consumer-groups.sh --describe). One partition lagging → hot key, poison pill, or a stuck/slow single consumer instance. All → capacity problem.
2. **Check rebalance churn:** logs for frequent 'Attempt to heartbeat failed'/'rebalancing' — max.poll.interval exceeded? Fix processing time or config; rebalancing groups make no progress.
3. **Measure processing:** time per record/batch; profile the handler — usually a slow DB call or sync HTTP call dominates. Batch/async it.
4. **Throughput math:** incoming msg/s vs (consumers × per-consumer rate). Scale consumers if below partition ceiling; else more partitions or two-stage topology.
5. **Check errors:** repeated retries on a poison record; DLQ it.
6. **External bottleneck:** downstream DB at capacity — consumer scaling won't help; fix the sink (bulk upserts, connection pool, index).
7. Meanwhile, confirm retention comfortably exceeds worst-case catch-up time so data isn't deleted unread.

Q72. Producer throughput suddenly drops and request latency spikes. What do you check?

1. **Broker health:** under-replicated partitions > 0? A broker down/slow shrinks ISR and (acks=all) stalls produces; check disk I/O saturation, GC pauses, network.
2. **min.insync.replicas violations:** NotEnoughReplicas errors mean ISR shrank below the floor — availability intentionally sacrificed; fix the broker.
3. **Producer buffer exhaustion:** buffer-available-bytes ≈ 0 and send() blocking on max.block.ms → downstream slowness backpressuring into the app.
4. **Batching regression:** a deploy changing keys/partitioner can shatter batching (records-per-request drops); check batch-size-avg and compression-rate metrics.
5. **Quotas:** throttle-time metrics > 0 → hitting client quotas.
6. **Leader skew:** after broker restarts, leadership may pile onto few brokers — run preferred leader election / rebalance.
7. **Network/DNS/TLS:** connection churn, handshake storms after an LB change.

Q73. How does Kafka achieve its performance? Summarize the key design choices.

1. **Append-only sequential I/O** to segment files — avoids random writes entirely.
2. **OS page cache as the cache:** no double-buffering in JVM heap; recent data served from memory, small heaps → tame GC.
3. **Zero-copy transfer** (sendfile) from page cache to socket for consumer fetches.
4. **Batching everywhere:** producer batches, broker stores batches as-is, consumers fetch large chunks.
5. **End-to-end compression** of whole batches with pass-through storage.
6. **Partitioned parallelism** spreading load across brokers and disks.
7. **Pull-based consumers** with long polling — no per-message broker bookkeeping of delivery state; consumers track offsets themselves, making a consumer nearly free for the broker.
8. Binary protocol with sparse index files rather than per-message indexes.

Q74. What is tiered storage in Kafka and why does it matter?

Tiered storage (KIP-405, production in Kafka 3.9+/vendors earlier) splits each partition's log into a **local hot tier** (recent segments on broker disks) and a **remote cold tier** (older segments offloaded to object storage like S3). Brokers fetch remote segments transparently when consumers read old offsets.

Why it matters:
1. **Long retention decoupled from broker disk:** months/years of history without giant expensive disks.
2. **Faster operations:** smaller local data → faster broker recovery, rebalancing, and partition reassignment.
3. **Elasticity:** scale compute (brokers) and storage independently.
4. Enables Kafka-as-source-of-truth / event sourcing economically. Trade-off: reading deep history is slower (object-store latency) and initially had limits (e.g., not for compacted topics).

Q75. How do you secure a Kafka cluster?

1. **Encryption in transit:** TLS on client-broker and inter-broker listeners (cost: loses zero-copy, some CPU).
2. **Authentication:** SASL — SCRAM-SHA-512 (username/password), OAUTHBEARER (OIDC tokens), GSSAPI/Kerberos (enterprise), or mTLS certificates.
3. **Authorization:** ACLs per principal on resources (topic/group/cluster) and operations (Read/Write/Create...), default-deny (allow.everyone.if.no.acl.found=false); prefix-based ACLs for team namespaces (payments.*).
4. **Encryption at rest:** disk/volume encryption (Kafka has no native at-rest encryption); field-level encryption client-side for PII.
5. **Hygiene:** separate listeners for internal/external traffic, quotas per principal, audit logging, secrets management for credentials, network policies restricting broker access.

Q76. How do you monitor Kafka in production? Name the metrics that actually matter.

Broker: UnderReplicatedPartitions (should be 0 — the single most important alert), OfflinePartitionsCount, ActiveControllerCount (exactly 1 cluster-wide), ISR shrink/expand rate, request handler idle ratio (<30% busy warning), disk usage, and produce/fetch p99 latencies.
Consumer: group lag (records AND time-lag — 'how many seconds behind'), rebalance frequency, commit failure rate.
Producer: record-error-rate, retry rate, buffer-available-bytes, batch-size-avg, request-latency-p99.
End-to-end: synthetic probe producing and consuming a canary topic measuring true e2e latency.
Stack: JMX → Prometheus (jmx_exporter) → Grafana + alerting; Burrow or exporter-based lag monitoring; Cruise Control for balance and self-healing.

Section 9: Architecture Patterns with Kafka

Q77. Explain the Saga pattern with Kafka (choreography vs orchestration).

A saga manages a distributed business transaction as a sequence of local transactions, each publishing events, with **compensating actions** instead of rollback.

Choreography: No central brain — services react to each other's events. OrderService emits OrderCreated → PaymentService reserves payment, emits PaymentReserved → InventoryService reserves stock... On failure, it emits a failure event and upstream services compensate (RefundPayment). Simple for short flows; becomes hard to reason about ('who reacts to what?') as steps grow, risks cyclic dependencies.

Orchestration: A saga orchestrator (e.g., OrderSaga service, possibly built on a state machine, Temporal, or Kafka-based state) sends commands to participants and consumes their replies, tracking saga state explicitly. Central visibility, easier timeouts/compensation logic; the orchestrator is extra infrastructure and a design hotspot.

Both need: idempotent participants, correlation (sagaId), timeouts with compensation, and events durable enough to survive crashes — Kafka's log fits naturally.

Q78. What is event sourcing and how does Kafka fit — and where does it fall short?

Event sourcing stores every state change as an immutable event; current state is a fold (replay) of events, optionally accelerated by snapshots. Benefits: full audit history, temporal queries, rebuildable projections, natural fit with CQRS.

Kafka's fit: durable ordered log, replay, compacted snapshot topics, consumers building projections.
Where it falls short as an event STORE:
1. No efficient 'load all events for aggregate X' — reading one entity's history means scanning a partition (unless one partition per aggregate, which doesn't scale).
2. No optimistic concurrency on append per aggregate ('append only if version = N'), which aggregates need to reject conflicting concurrent commands.

Common architecture: a real event store or DB (even the outbox) as the write-side source of truth per aggregate, with Kafka as the **distribution backbone** feeding projections, caches, and other services.

Q79. Explain CQRS and how Kafka enables it.

CQRS separates the **write model** (commands, normalized, consistency-focused) from **read models** (queries, denormalized, purpose-built). Kafka is the spine: the write side persists changes and publishes events (outbox/CDC); independent projector consumers build each read model — Elasticsearch for search, Redis for hot lookups, a warehouse for analytics — each in its own consumer group, each rebuildable by replaying the topic from earliest.

Costs to acknowledge in interviews: eventual consistency between write and read sides (UI strategies: read-your-writes from the write model, optimistic UI, or version-checked reads), duplicated storage, and projection versioning/rebuild pipelines as schema evolves.

Q80. Design a notification system (email/SMS/push) using Kafka.

1. **Ingestion:** Services publish NotificationRequested events (userId key for per-user ordering, channel-agnostic payload) to a notifications topic — decoupling senders from delivery.
2. **Preference & routing service:** Consumes requests, applies user preferences/opt-outs, templates content, fans out to per-channel topics: notifications.email, notifications.sms, notifications.push (channels scale and fail independently).
3. **Channel workers:** Consumer groups per channel call providers (SES, Twilio, FCM) with bounded retries + exponential backoff; provider rate limits enforced via consumer pacing/quotas; failures → channel DLQs; provider callbacks/webhooks publish DeliveryStatus events.
4. **Idempotency:** notificationId dedup table so redeliveries don't double-send.
5. **Priority:** separate topics (transactional vs marketing) so OTPs never queue behind campaigns.
6. **Observability:** status topic powers per-user notification history and delivery-rate dashboards.

Q81. How would you migrate a synchronous REST-based flow to Kafka without downtime?

1. **Strangler + dual publish:** Keep the REST call, additionally publish the event (via outbox to avoid dual-write). Downstream consumers build against real production traffic in shadow mode; compare outcomes with the sync path.
2. **Cutover per consumer:** Once a downstream service's async path is verified (metrics parity, reconciliation checks), stop calling it synchronously; it now reacts to events. Feature-flag the switch for instant rollback.
3. **Contract discipline:** Schema registry with BACKWARD compatibility from day one.
4. **Handle semantics change:** Callers previously got confirmation synchronously — either return 202 Accepted with a status endpoint/webhook, or keep the critical validation synchronous and move only side effects async.
5. **Retire** the REST path after a bake period; keep replay ability as the new-consumer onboarding story.

Q82. When should you NOT use Kafka?

1. **Simple task queues at modest scale:** RabbitMQ/SQS are simpler to run and offer per-message ack/retry/delay natively.
2. **Request-response RPC:**Synchronous APIs are simpler and lower-latency; messaging adds correlation machinery.
3. **Delayed/scheduled delivery & per-message TTL:** Not native to Kafka (needs retry-topic tiers or external schedulers); RabbitMQ has these built in.
4. **Strict global ordering across all messages:** Forces one partition → no parallelism.
5. **Large binary transfer:** Use object storage + claim check.
6. **Very small teams / low volume:** Cluster ops burden (or managed-service cost) outweighs benefits when a Postgres queue table (SKIP LOCKED) would do.
7. **Consumer-selective routing on message content:** Broker-side filtering/routing is RabbitMQ's strength; Kafka consumers read whole partitions.

Section 10: Kafka vs Other Systems

Q83. Kafka vs RabbitMQ — give a decision-oriented comparison.

Model: Kafka = replayable distributed log, consumers pull and track offsets; RabbitMQ = smart broker routing messages through exchanges/bindings to queues, push with per-message acks, deleted after consumption (streams feature aside).
Choose Kafka for: high-throughput event streaming (multi-GB/s), multiple independent consumer groups, replay/audit, stream processing, event sourcing/CDC backbones, per-key ordering at scale.
Choose RabbitMQ for: task/work queues with per-message retry & priority, complex routing (topic/headers exchanges), delayed messages & TTLs, request-reply RPC conveniences, lower operational footprint at modest scale.
Ordering: Kafka per partition; RabbitMQ per queue but broken by redeliveries/competing consumers.
Retention: Kafka time/size-based regardless of consumption; RabbitMQ consumption-based.
Interview one-liner: 'RabbitMQ moves tasks; Kafka records facts.'

Q84. Kafka vs AWS SQS/SNS — when is each appropriate?

SQS (queue) + **SNS** (fan-out pub-sub) are fully managed, near-zero-ops, pay-per-use, effectively infinitely elastic. SQS gives per-message visibility timeout, DLQ, delay queues; FIFO queues give ordering + exactly-once-processing per message group at capped throughput.
Kafka advantages: replay & retention (SQS deletes on consume; no rewind), multiple consumer groups over the same data, strict per-key ordering at high throughput, stream processing ecosystem (Streams/Flink/Connect), lower per-message cost at sustained massive volume, no 256KB message cap.
Rule of thumb: AWS-native app needing decoupled task processing → SQS/SNS (or EventBridge for routing). Data/event platform, replayable source of truth, streaming analytics → Kafka (or MSK/Confluent Cloud to keep it managed).

Q85. Kafka vs Apache Pulsar — key architectural differences?

Pulsar separates compute and storage: stateless brokers serve traffic; Apache BookKeeper 'bookies' store segments. Benefits: instant broker scaling (no data movement), fast failover, native tiered storage early, built-in multi-tenancy and geo-replication, and flexible subscription modes (exclusive/shared/failover/key_shared) that give per-message queue semantics AND streaming from one system.
Kafka: brokers own storage (though tiered storage + KRaft narrowed gaps), a far larger ecosystem (Connect, Streams, ksqlDB, tooling, talent), simpler architecture (one system vs brokers+bookies+ZK/quorum).
Practical take: Pulsar shines for multi-tenant, geo-replicated, mixed queue+stream workloads; Kafka wins on ecosystem maturity, community, and hiring — the default unless a Pulsar-specific strength drives the decision.

Q86. Kafka vs Redis Pub/Sub and Redis Streams?

Redis Pub/Sub: Fire-and-forget broadcast — no persistence, no replay; disconnected subscribers miss messages permanently. Only for ephemeral signals (cache invalidation pings, live presence).
Redis Streams: Persistent append-only log with consumer groups, acks (XACK), and pending-entry claims — genuinely queue-like, in-memory fast, great when Redis is already deployed and volume/retention fit in memory.
Kafka over Redis Streams when: data exceeds memory economics, long retention/replay is needed, multi-broker partitioned scaling and replication guarantees matter, or the Connect/Streams ecosystem is wanted. Redis wins on simplicity and sub-millisecond latency for small-scale internal pipelines.

Q87. What are Kafka Streams vs Flink vs Spark Structured Streaming trade-offs?

Kafka Streams: A library embedded in your service — no separate cluster; scales with app instances; EOS with Kafka; best for Kafka-to-Kafka microservice-style processing with moderate state. Limited to Kafka sources/sinks.
Flink: A dedicated cluster; true event-time streaming engine with the richest semantics — huge keyed state, savepoints, complex event processing, async I/O, many connectors; the choice for large-scale, complex, low-latency pipelines. Higher ops cost.
Spark Structured Streaming: Micro-batch (higher latency, though continuous mode exists); shines when teams already run Spark for batch and want unified batch+stream code for analytics/ETL into lakes.

Heuristic: app-embedded transformations → Kafka Streams; demanding streaming platform → Flink; Spark shop doing analytics → Spark.

Section 11: Scenario-Based & Behavioral-Style Questions

Q88. Scenario: After adding partitions to a keyed topic, per-entity ordering broke. Explain why and how to fix it.

Why: Partition = hash(key) % N. Changing N remaps most keys to different partitions. Old events for key K sit in partition 2 while new events land in partition 5; consumers process the two partitions independently, so a newer event can be handled before an older one — per-key ordering is violated across the boundary.

Fixes / mitigations:
1. **Drain-then-switch:** Pause producers, let consumers fully drain, then resume on the new count (brief pause, clean boundary).
2. **New topic migration:** Create orders-v2 with the target partition count; dual-write or replay from v1 preserving key order; cut consumers over at a coordinated offset.
3. **Version-aware consumers:** tolerate reordering via per-entity sequence numbers, buffering out-of-order events briefly.
4. **Prevention:** over-provision partitions initially; treat partition count of keyed topics as effectively immutable.

Q89. Scenario: A consumer processes a message, writes to the DB, then crashes before committing the offset. What happens and how do you make it safe?

On restart (or after rebalance), the partition's committed offset still points at that message — it is **redelivered** and the DB write executes again: the classic at-least-once duplicate.

Safe designs:
1. **Idempotent write:** Upsert keyed by business/event ID, or INSERT ... ON CONFLICT DO NOTHING against a unique constraint — the duplicate becomes a no-op.
2. **Dedup table:** In the same DB transaction as the write, insert eventId into processed_events (unique); redelivery hits the constraint and is skipped.
3. **Offsets in the DB:** Store (partition, offset) in the same transaction as the state change; on startup, seek() to DB offsets — offset and effect are atomic, eliminating the gap entirely.
Never solve this by committing before processing — that converts duplicates into data loss.

Q90. Scenario: One partition has 10x the traffic of others (hot partition). Diagnose and fix.

Diagnosis: Per-partition metrics (messages-in, bytes-in, consumer lag by partition) reveal skew; almost always a **hot key** — one tenant/celebrity user/default value (null key with old sticky bug, or 'UNKNOWN' IDs) hashing to one partition.

Fixes:
1. **Composite keying:** key = entityId + '-' + (hash(subId) % k) spreads a hot entity across k partitions — sacrificing total order per entity for order per sub-stream; valid only if strict per-entity order isn't required (or can be restored downstream by sequence numbers).
2. **Fix degenerate keys:** stop keying on low-cardinality fields (country, status) — key on the true entity.
3. **Two-tier topics:** route whale tenants to a dedicated topic/partition set with dedicated consumers.
4. **Local aggregation:** pre-aggregate at the producer to cut the hot key's message rate.
5. Scale the single consumer stuck with the hot partition vertically (it can't be parallelized by adding members).

Q91. Scenario: Your team needs to reprocess the last 7 days of events after fixing a bug in a consumer. How do you do it safely?

1. **Verify retention** covers 7 days on the topic (retention.ms) — if not, restore from a sink/archive (S3 via Connect) instead.
2. **Decide the mechanism:**
- **Same group reset:** Stop all consumers in the group (mandatory — reset fails on active groups), run kafka-consumer-groups.sh --reset-offsets --to-datetime --execute, restart. The group reprocesses in place.
- **Parallel replay group:** Deploy the fixed consumer under a NEW group.id starting at the timestamp, writing corrected results; keeps the live path running — often safer.
3. **Idempotency check first:** Reprocessing re-executes side effects — ensure upserts/dedup so 7 days of duplicates don't double-charge/double-email; disable non-idempotent side effects (notifications) during replay if needed.
4. **Throttle & monitor:** replay competes with live traffic for broker I/O and downstream DB capacity; watch lag on unrelated consumers.
5. **Reconcile:** row counts / checksums between expected and reprocessed outputs before declaring success.

Q92. Scenario: Rolling deployment of a 50-instance consumer group causes minutes of processing stalls. Fix it.

Each pod restart triggers leave+join rebalances; with the eager protocol the whole group stops on every one — 50 instances × 2 rebalances = a storm.

Fixes (combine all):
1. **CooperativeStickyAssignor:** incremental rebalancing — untouched partitions keep processing.
2. **Static membership** (group.instance.id per pod, e.g., StatefulSet ordinal) + session.timeout.ms raised to ~2–5 min: a pod bouncing within the window reclaims its partitions with NO rebalance.
3. **Graceful shutdown:** handle SIGTERM → stop polling, finish in-flight batch, commit offsets, close() cleanly (close sends LeaveGroup; with static membership it deliberately doesn't, which is what you want).
4. **Slow the rollout** (maxUnavailable=1, readiness gates) so membership changes don't overlap.
5. Verify max.poll.interval.ms isn't also ejecting members mid-deploy due to cold-start slowness (warm caches before subscribing).

Q93. Scenario: Product asks for 'guaranteed delivery within 5 seconds' for order events. What do you push back on and what do you build?

Push back / clarify: 'Guaranteed' and 'within 5s' are two different SLOs. Delivery guarantee = no loss (durability); latency SLO = p99 end-to-end time, which cannot be absolutely guaranteed during broker failover, consumer rebalance, or downstream outage — only met with high probability and monitored.

Build:
1. **Durability:** acks=all, idempotent producer, RF=3, min.insync.replicas=2, outbox at the source.
2. **Latency:** linger.ms small (0–5), adequate partitions/consumers, processing kept async and lightweight, no blocking calls in the poll loop; cooperative rebalancing + static membership to shrink stall windows.
3. **Measure:** event-time to processed-time histogram (true e2e), alert on p99 > 5s and on lag-in-seconds.
4. **Degradation story:** what happens beyond 5s — is late processing acceptable (usually yes) or must it trigger a fallback? Encode that explicitly rather than pretending failures won't happen.

Q94. Scenario: You suspect messages are being lost between producer and consumer. List every place loss can occur and the fix.

1. **Producer fire-and-forget:** send() without checking the future/callback → silent loss on error. Fix: handle callbacks, alert on record-error-rate.
2. **acks=0/1:** leader dies pre-replication → acked data gone. Fix: acks=all + min.insync.replicas=2 + RF=3.
3. **Producer buffer drop on shutdown:** process exits without flush()/close() → buffered records lost. Fix: graceful shutdown hooks.
4. **delivery.timeout exhausted** during long broker outage → callback error possibly ignored. Fix: failure handling + fallback persistence.
5. **Unclean leader election enabled:** out-of-sync leader loses committed data. Fix: keep it false.
6. **Retention expiry before consumption:** lagging consumer + short retention → offsets jump (auto.offset.reset=latest silently skips). Fix: retention ■ max expected lag; alert on lag-vs-retention; auto.offset.reset=none for critical apps to fail loudly.
7. **Consumer commits before processing** (auto-commit patterns) → crash loses the batch. Fix: commit after processing.
8. **Swallowed exceptions in handler:** record 'processed' but actually dropped. Fix: error handling discipline + DLQ.

Then verify with an end-to-end reconciliation (counts/checksums by hour).

Q95. Scenario: Design the event backbone for an e-commerce order flow (order, payment, inventory, shipping, notification).

Topics (domain events, keyed by orderId): orders.events, payments.events, inventory.events, shipping.events — one topic per aggregate/domain, not per event type; event 'type' in the payload/headers. RF=3, min.insync.replicas=2, partitions sized for peak (e.g., 24–48), 7–30d retention, schema registry with BACKWARD compatibility.

Flow (choreography saga): OrderService writes order + outbox row atomically → OrderCreated. PaymentService consumes, charges (idempotency key = orderId+attempt), emits PaymentCaptured/PaymentFailed. InventoryService reserves stock on PaymentCaptured, emits StockReserved/OutOfStock. On failure events, upstream services compensate (refund, release) — every consumer idempotent via dedup on eventId. ShippingService and NotificationService subscribe independently (own consumer groups).

Cross-cutting: correlationId=orderId in headers + tracing; DLQ per consumer with replay tooling; lag & DLQ-depth alerts; order status read model (CQRS projection) for the UI, accepting eventual consistency with optimistic UI on placement.